Abstract

This project is concerned with the harmonic analysis of four part chorales written by J.S. Bach which leads to removing all non-harmonic tones and dissonances which feature within. This report is not only aimed at music enthusiasts, as any concepts introduced are explained throughout. The approach is implemented in the Python programming language and seeks to detect simultaneously occurring notes in a music score in order to determine the notes which should be removed. The final outcome was positive and indicates a successful beginning in the attempt to achieve the aims of the project. Clear objectives have been discovered and explain what has to be pursued next in order to improve the system in the future and allow it to become 100% accurate.

Contents

1 Introduction **3**
- 1.1 Bach's Chorales for Four Parts 3
- 1.2 What is a Non-Harmonic Tone? 3
- 1.3 Existing File Structure 4

2 Extraction of Data **6**
- 2.1 XML Parsing in Python 6
- 2.2 Structure of a MusicXML File 6
- 2.3 Selecting the Required Data 8

3 The Time-Slice Operation **10**
- 3.1 Issues with Time in MusicXML 10
- 3.2 Taking Vertical Slices 11
- 3.3 Recovering Original Data 13

4 Chord Classification **14**
- 4.1 Defining a List of Chords 14
- 4.2 Chord Detection Techniques 15
- 4.3 Removal of Non-Harmonic Tones 16
- 4.4 Ambiguity in Results 17

5 Output as Musical Score **18**
- 5.1 Compressing the Time Scale 18
- 5.2 Write to XML File 18
- 5.3 Output via Lilypond 19

6 Further Considerations & Improvements **20**
- 6.1 Key Signatures & Modulation 20
- 6.2 Harmonic Rhythm and Cadences 20

7 Conclusion **22**

Appendices **23**
- Appendix A 24
- Appendix B 25
- Appendix C 27
- Appendix D 28
- Appendix E 29
- Appendix F 30
- Appendix G 31
- Appendix H 32

Bibliography **33**

1 Introduction

The aim of this project is to be able to perform harmonic and intervallic analysis on four part chorales written by J.S. Bach to remove all non-harmonic tones from the scores. The details of why and how this project is being undertaken are featured in this section.

1.1 Bach's Chorales for Four Parts

Bach is very much regarded a master of harmony [15], and his chorales written for four parts are no different. In fact they had such a huge impact in the world that they are still used to this day to teach the methods of harmonic analysis. This analysis is not always easy, even the most experienced of musicians will have to sit down and think hard before passing their judgement, and even then there is often disagreement over the true answer.

For this reason, it would be very beneficial to implement a program that can systematically test these chorales and produce a harmonic analysis which can identify all chordal structures and therefore all non-harmonic tones present in the piece.

1.2 What is a Non-Harmonic Tone?

Before attempting to remove a non-harmonic tone, it would perhaps be better to start by describing exactly what one is. There are many different types of non-harmonic tone however it simply can be described as any note which is not part of the chord that is being played. [18] This is to say that all notes that create any sort of dissonance are determined as non-harmonic. Removing these notes is absolutely not beneficial to the musical score, as it would degrade the sense of flow and all the decorations that make music interesting. However, as mentioned above, performing this exercise can provide a valuable lesson in harmony.

It would be best to describe these notes with musical examples, with the first shown in ***Figure 1a***.



G	(G)	D
D	E	D
B	B	A
D	D	F#
G	G	D

Figure 1a *– Musical score with four voices (left) and tabled version showing exactly what notes are being played, and what the overlying chord implication is (right). The non-harmonic tone is circled in red, and parentheses show that a chord is still implied despite the dissonant tone included.*

In this example, the note E creates a dissonance with the G chord, and is therefore non-harmonic. Without its presence the movement from the G → D chord would be very static and uninteresting.

Figure 1b shows another example below.

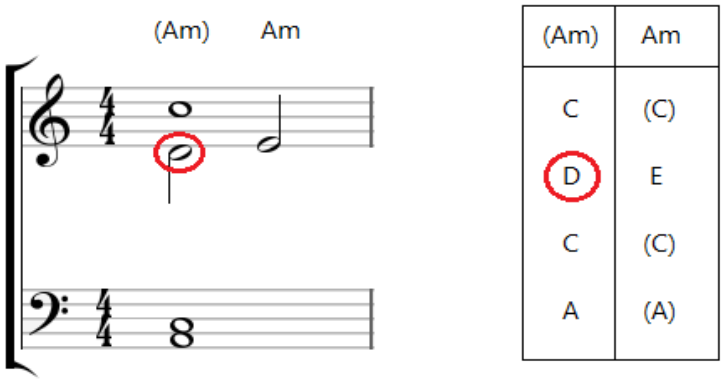


Figure 1b *– Another music score(left) and its tabled equivalent (right). The first chord is still implied to be an A Minor, as the non-harmonic tone resolves to complete the chord.*

Here it is slightly different as the A Minor chord is not completed until the 3rd beat of the bar. However, the first chord is still implied to be an A Minor chord due to the resolution of the D → E, whilst all other notes are suspended.

Figure 1c shows the final example.

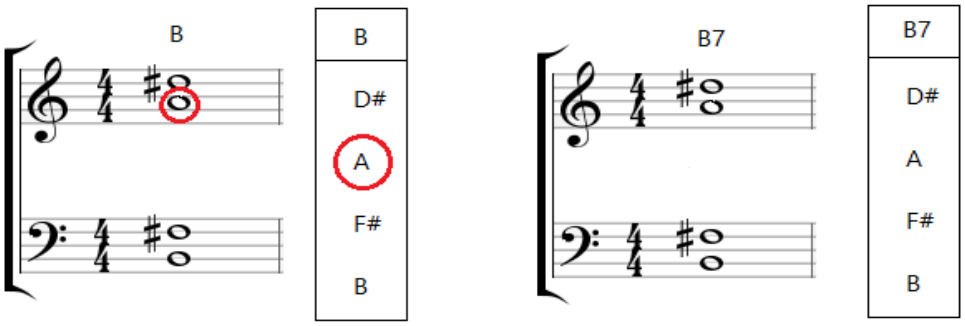


Figure 1c *– The same score and table (left and right), however with different results for the non-harmonic tones.*

This is a specific example when a 7th note is added to a chord. The note can be considered either harmonic or non-harmonic, depending on the key of the piece. Obviously this creates some room for objectivity when deciding whether notes are the former or the latter. This idea will be explored later on in Section 5.

1.3 Existing File Structure

Provided at the start of the project was a set of pre-existing materials that had previously been used to perform different types of analysis on similar pieces of music. The materials consisted of a PostgreSQL [7] database structure which made use of functions coded in Python [10] in order to store and manipulate the information contained within musical scores. Specifically, new data types were created in a "spiral of fifths" [17] (or Spoff) format which would be particularly useful for this project as they contain functions to perform calculations on both pitches and intervals. Slightly

reduced versions of these functions are included in the final submitted code, in order to provide means of calculating intervals.

However it was decided that this existing database structure would not be used. Instead the entire system would be designed using Python. This decision was made due to the fact that the input files for the previous project were highly modified versions of the XML files provided for this project, which were not available. It was decided instead to pursue a means of extracting the data solely from the XML files as they are already a standard interface for exchanging and storing musical information.

2 Extraction of Data

Provided at the start of the project were a set of XML (Extensible Markup Language) [11] files which contain the score information for chorales written by J.S. Bach, particularly those which were written for an SATB choir (four unique voices, soprano, alto, tenor and bass). [13] This section details the processes of extracting the relevant information from these files in order to be able to analyse the musical scores. The final result is a list of all notes present in the score, each with their own associated information stored.

2.1 XML Parsing in Python

In order to be able to extract information from an XML file, an appropriate library [8] would have to be used which can parse the data within Python. This basically means giving the code some way to understand the structure and syntax of a different file type, and then being able to store the data in its own language to be manipulated more easily. Many libraries exist which can perform this function, each with their own advantages and disadvantages, with some of the possibilities listed below;

- ElementTree [2]
- lxml [4]
- Minidom [5]
- BeautifulSoup [1]

ElementTree and Minidom are the only two of these which are available in the default standard libraries included with Python 2.7, so their use over the other two was investigated first as it was possible to immediately start attempting to parse some information from the files. ElementTree seems to be widely regarded as the best way to go about doing this. However it was decided to instead use Minidom due to its simpler syntax and widely available documentation. It can sometimes be regarded as quite a memory-heavy process, however with the relatively small size of the XML files provided it did not seem to have any negative impact on the time taken to complete the task.

Another big advantage is the requirement Minidom has with stepping through each tag of an XML file in accordance with its hierarchy. While this may seem standard for any XML parsing tool, it is particularly useful for the structure of the files which were provided as is explained in the next section.

2.2 Structure of a MusicXML File

The XML files provided are actually of a unique structure and layout, and these are referred to as MusicXML[6] files. The basic hierarchical structure is shown in ***Figure 2a***, and it can be seen that each file is essentially a tiered system of tags. The data required to represent a Tier 2 tag for example is the combined information from Tier 3, Tier 4 and Tier 5. So it can be said that the tag from the outermost tier requires all the information from all inner tiers before the tag can be closed. To put this in musical terms it means that to make up a Tier 3 tag (i.e. a note) you require the pitch information: a step value and an octave value, and the duration information. This feature is recursive until all notes in the score have been accounted for, which shows why it is good to have Minidom step through the file in order to extract all the information from each bar in the first voice,

before getting the information from the second voice, and so on. The XML hierarchy tree is shown in ***Figure 2a***, and an actual example of XML code shown in ***Figure 2b***.

Tier 1	Tier 2	Tier 3	Tier 4	Tier 5
<part>				
	<measure>			
		<note>		
			<pitch>	
				<step>
				</step>
				<octave>
				</octave>
			</pitch>	
			<duration>	
			</duration>	
		</note>		
		<note>		
			...	...
			...	
		</note>		
		...		
	</measure>			
	<measure>			
		...		
	</measure>			
	...			
</part>				
<part>				
	...			
</part>				
...				

Figure 2a *– XML hierarchy tree highlighting the different tiers present in the file as described in Section 2.1.*

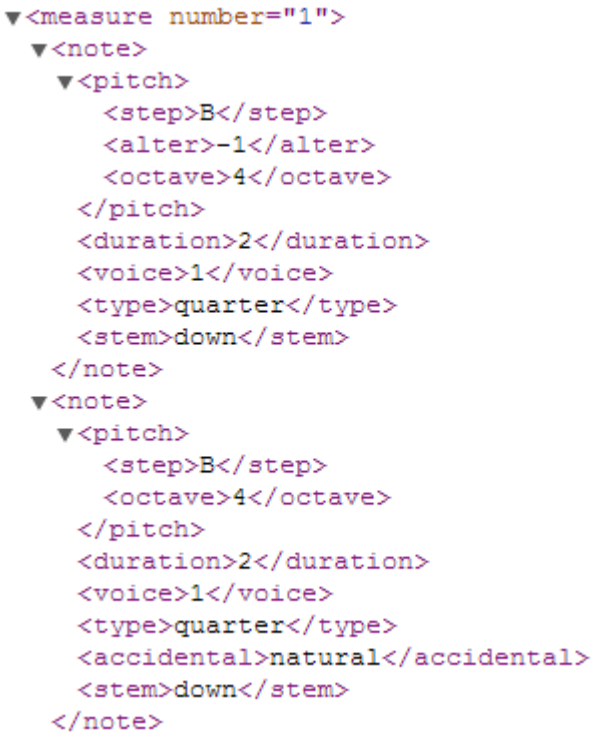


Figure 2b *– Example MusicXML code showing the data contained within each tag. This picture shows a small section between Tiers 2 and 5.*

2.3 Selecting the Required Data

Now that the file structure has been explored, the correct information has to be chosen to be extracted. Quite a lot of separate information for each note has to be stored, which isn't a big problem as Python is naturally very fast and efficient at handling lists. There is a data type called a dictionary [9] which allows the storing of multiple values under different key names, as shown below;

note = {'step': G, 'octave': 4, 'measure': 5, etc..}

So for each note that occurs, a dictionary will be used to store multiple pieces of information from the required XML tags. This means that the final result of the extraction will be a list of dictionaries, with the length being equal to the number of notes present in the piece. Using the Minidom library, the tiers of the XML tree were iteratively stepped through so that the data would be stored in the correct order. A typical example of a single dictionary entry is shown in ***Figure 2c*** with eight unique variables being stored per note. A description of each variable is shown below;

Part *: The current part number (Voice 1, Voice 2, etc...)*
Measure*: The current bar number (Bar 1, Bar 2, etc...)*
Step*: The pitch class of the current note (A, B, C, D, E, F, G)*
Octave*: The octave number of the current note, C4 is middle C (3, 4, 5, etc...)*
Duration*: The length of the current note (1, 2, 4, 8, etc...)*
Alter*: Determines whether the note contains an accidental (-1 : flat, 0 : none, 1 : sharp)*

***Pitch**: A calculation based on the Spoff types discussed in Section 1 (2, 4, -6, 13, etc...)*
***Divisions**: The number of divisions present within this part (2, 4, 8, etc...)*

Tag	Value
Part	2
Measure	5
Step	G
Octave	4
Duration	2
Alter	0
Pitch	2
Divisions	2

***Figure 2c** – Typical entry(dictionary) in the stored list of notes. Need to record these eight variables for each individual note in the score.*

So once the list has been compiled with all the relevant data, it can simply be split in four so that there is now an ordered list of notes for each voice present in the score of music. All the variables recorded should be all that is required to perform harmonic and intervallic analysis of the piece later on.

3 The Time-Slice Operation

This section deals with the issue of time in a MusicXML file. Operations have to be applied to the parsed data in order for it to retain its musical clarity. Otherwise it will be hard to perform any kind of analysis on the data in accordance with time. These operations and the option to reverse them are detailed below.

3.1 Issues with Time in MusicXML

As mentioned in Section 2.2, the structure of a MusicXML file does not allow any way to infer meaning from the time information provided for a music score. [17] That is to say it gives a note duration value, and a corresponding number of divisions per bar specific to each voice. These two pieces of information are only useful if all previous note durations from one voice have been taken into account and summed together, and even then they will not align musically with these same durations for another voice.

The way in which the score data is being stored after having parsed the XML file is simply an ordered list of each note and all of its unique attributes. It can easily be seen in ***Figure 3a*** how this method of storing the data is incorrect musically, for example by taking the first seven notes present in Voice 1 and comparing them against the first seven notes present in Voice 2 there is a clear discrepancy in the amount of time which passes before each set of notes has been completed.



	Two bars, divided into 8 sub-beats each, 16 total															
Voice 1:	1		2		3		4		5				6		7	
Voice 2:	1	2	3	4	5	6	7		8		9		10	11	12	13

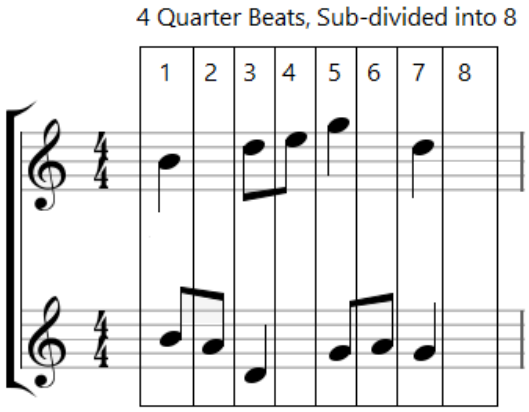
Figure 3a *– Two bars of music for Voice 1 and Voice 2, numbers above score simply represent the number of notes sang in each part(top) and tabled version of these numbers showing at which point in time they are being sang(bottom).*

For this reason, it is very evident that another method of storing the data would have to be found so that notes in multiple voices could be compared to each other in accordance to the time in which

they occur in the score, as opposed to simply recording the order they are played.

3.2 Taking Vertical Slices

In order to be able to manipulate the data present in a score, the goal is to be able to take accurate vertical slices at any point in time. In ***Figure 3b***, a simple bar of two intertwining melodies is shown. Because of the differences in note durations, the two parts do not move in parallel motion (i.e. at the same time). Instead there is an overlap as new notes occur in one voice where a note is still being held by the opposite voice.



Beat:	1	2	3	4	5	6	7	8
Voice 1:	B	(B)	D	E	G	(G)	D	(D)
Voice 2:	B	A	D	(A)	G	A	G	(G)

Figure 3b *– Typical score with simple overlapping motion between voices (left) and tabled representation of notes which occur at each of the 8 sub-divisions(right). Parenthesis indicates the note is being held on from a previous beat.*

It is desired for the computer to be able to recognise these overlapping notes and treat them as if the note which is being held has instead just been played. For example on the second beat, the information should be looked at and determined that notes of the pitch classes A and B are occurring simultaneously, despite the fact that the B note is not triggered at this point in time, it is still harmonically implied by the suspension from the previous beat. It should be noted that not all of these simultaneities are declared as chords, they are simply a collection of notes which may or may not imply a new chord occurs on this beat.

In order to do this it is required to elongate the lists which store the musical information so that the number of entries in each voice is identical. If two voices are identical lengths then it is possible to equate each individual index value of these lists to a unique time event within a piece of music. This is to say for example, that taking the fifth index value of both lists would return the two notes which are harmonically implied (regardless of whether they are triggered at this point or not) at this fifth beat along the musical time-line. However, this fifth beat along does not hold any real meaning without some sort of segmentation of each bar of the piece.

To maintain the clarity of the original score whilst elongating the lists, it is necessary to divide each bar into small sections, as mentioned above. The way this is done is by first searching for the shortest note length present in the entire score. Knowing the shortest length gives the correct number of segments to split each bar into. For example, if the shortest occurring note was a quaver($1/8^{th}$ note) then each bar should be split into eight equal sections. This allows for analysis of the score at every point where a new note is introduced, and every point in between without missing out any important information. So to take the example previously where the fifth index value is being inspected, in this case it would return the note information that occurs 5/8ths into the first bar.

Now all that remains to do is ensure this elongation will accurately represent the original score. Keeping with the example of eight segments per bar, it is now essential that the information stored holds eight entries for each bar. So if one bar contains four crotchets (1/4 note) then each note would simply be stored twice, giving the impression that it is now a piece of music with eight quavers. Higher note values like the minim(1/2 note) or semibreve(whole note) would simply be stored four times and eight times respectively, and anything which is already a quaver, would of course just be stored once. ***Figure 3c*** highlights this in an example with the original simple two bar melody, and the representation of what the piece of music should now look like after having this operation applied to it.

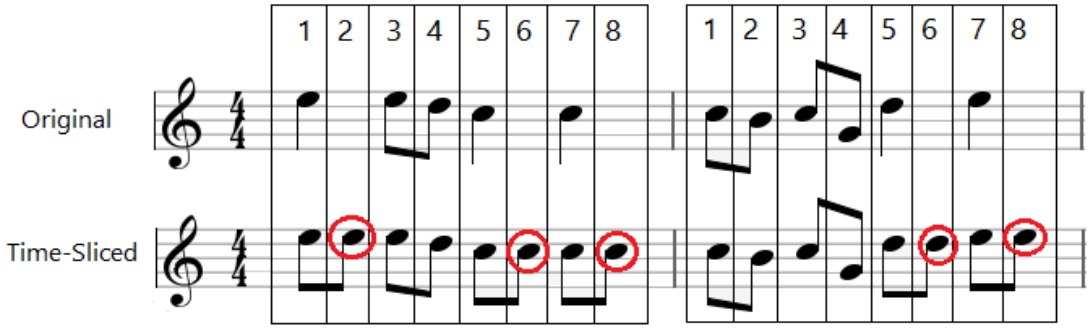


Figure 3c *– A two bar melody in its original form, and the new form after having been manipulated. Notes circled in red do not occur in the original piece and have been added to split each bar into eight segments. The red note value is always the same as the previous note as it is just the note which is still occurring after having been triggered one segment previously.*

Of course this same method can be applied to a higher number of divisions per bar. If the shortest value of note in a piece of music is a semiquaver (1/16th note) then each bar is therefore segmented sixteen times. A slightly more complex example is shown in ***Figure 3d***, again showing an original melody and the new version which represents how the code is storing the information.

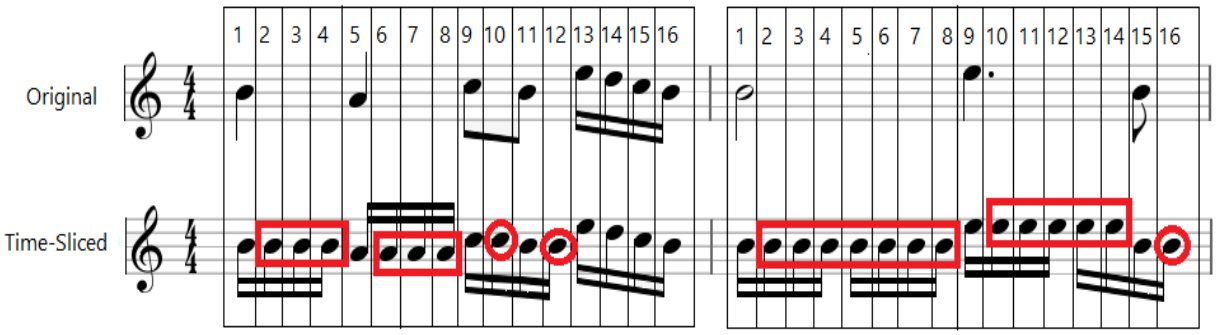


Figure 3d *– Two bar melody this time with sixteen divisions per bar. Notes highlighted in red again don't actually occur at this point in time in the music but their note value is still being implied at this point.*

The overall result of performing this time-slice operation is a longer list of information for each

voice, which allows the harmonic structure at any point in the music to be pinpointed exactly. So a score which has 10 bars, with eight divisions per bar, will store 8 x 10 = 80 notes per voice, which can all be referred to simply by the index value of each list.

3.3 Recovering Original Data

Of course when modifying the data stored in quite an extreme way like this it is very important to add in a feature which will allow the original score information to be recovered after modification. This is achieved by assigning a new piece of information to each entry in a list, a flag. This flag will contain either a "Yes" or a "No" which will indicate if the note was present in the original piece of music or not. So when duplicating entries when elongating the original list, all duplicates will be assigned a value of "No", whereas the original notes will be given a value of "Yes". ***Figure 3e*** illustrates an example of this.



Note:	D	D	C	B	E	E	C	B
Flag:	Yes	No	Yes	Yes	Yes	No	Yes	Yes

Figure 3e *– One bar melody and its time-sliced representation(top) and a tabled version of the time-sliced representation(bottom) showing exactly which notes will be given a flag value of "Yes". This shows which notes were present in the original score.*

Again, for further clarity an example is shown with a melody which contains 16 divisions per bar in ***Figure 3f***.



Note:	B	B	B	B	D	C	E	E	B	B	B	B	B	B	B	B
Flag:	Yes	No	No	No	Yes	Yes	Yes	No	Yes	No	No	No	No	No	No	No

Figure 3f *– One bar melody and its time-sliced representation(top) and a tabled version of the time-sliced representation(bottom). It is easy to see in this example that the number of "No" values far outweighs the "Yes" values as there are only 5 individual notes present in the original melody.*

4 Chord Classification

Now that is possible to take vertical slices of a musical score at any point in time as described in the previous section, a method has to be implemented that will allow the harmonic implication of these simultaneously occurring notes to be determined. Given a set of four vertically sliced notes from a Bach chorale, it is possible to use existing functions [17] to calculate the intervals present between notes and thus build from this result some kind of structure which resembles the shape of a chord.

4.1 Defining a List of Chords

Initially a set of pre-existing chordal structures must be defined so that comparisons can be drawn from the interval results and matched to a chord type. Given that there is a maximum of four voices simultaneously being sang in a four part chorale, there are only so many chords which can be represented.[14] The most commonly occurring chords are triads (chords made up of three unique pitch classes) which would obviously leave one leftover note. This leftover note has to be doubled in order to retain the harmony of the original sound. For example a G Major triad could be made up of the notes G, B and D with an additional G added to emphasise the root of the chord. The root and the fifth of the chord are the most commonly doubled notes with the third only being doubled in very specific cases. It should also be mentioned that not all three notes in a triad need to be present, however in this case only the fifth of the chord may be removed, as the third is what determines whether the chord is major or minor. So the set of notes G, G, B and B would still be called a G Major chord, however G, G, D and D would not as its overall tonality would be unclear.

However there are of course alternatives to doubling notes, one very simple example being when the root is tripled (i.e. G, G, G, B – still a G Major). The other option is introducing a fourth unique pitch class which opens the door to many new chords. Many of these contain notes which would be defined as non-harmonic, however they had initially been considered to gauge the kind of chords Bach used in his chorales. ***Figure 4a*** shows a list of some chords which can be made up of four unique pitch classes, and the intervals required to make them.

Chord Type	Intervals
Major	P1 – M3 – P5
Minor	P1 – m3 – P5
Augmented	P1 – M3 – m6
Diminished	P1 – m3 – m5
Suspended 4th	P1 – P4 – P5
Suspended 2nd	P1 – M2 – P5
Flat Five	P1 – M3 – m5
Major Sixth	P1 – M3 – P5 – M6
Minor Sixth	P1 – m3 – P5 – M6
Dominant Seventh	P1 – M3 – P5 – m7
Major Seventh	P1 – M3 – P5 – M7
Minor Seventh	P1 – m3 – P5 – m7
Augmented Seventh	P1 – M3 – m6 – m7
Diminished Seventh	P1 – m3 – m5 – M6
Seventh Flat Five	P1 – M3 – m5 – m7
Minor Seventh Flat Five	P1 – m3 – m5 – m7
Add Nine	P1 – M3 – P5 – M9

Key
P1 = Perfect Unison
M2 = Major Second
m3 = Minor Third
M3 = Major Third
P4 = Perfect Fourth
P5 = Perfect Fifth
m6 = Minor Sixth
M6 = Major Sixth
m7 = Minor Seventh
M7 = Major Seventh
M9 = Major Ninth

Figure 4a *– Table showing chord type and the intervals which make up these chord types(left) and a key to show what each intervallic property means(right).*

Once some pieces had been analysed, it was determined that only some of the chord types from ***Figure 4a*** occurred regularly and contained no dissonance. This new and reduced list of chord types is shown in ***Figure 4b***.

Chord Type	Intervals
Major	P1 – M3 – P5
Minor	P1 – m3 – P5
Dominant Seventh	P1 – M3 – P5 – m7
Major Seventh	P1 – M3 – P5 – M7
Minor Seventh	P1 – m3 – P5 – m7

Figure 4b – *New list of commonly occurring chord types considered to be fully consonant.*

So to summarise, if there is no doubling present, then the chord must have four separate pitch classes to form a new chord (possibly containing a non-harmonic tone), or there must be a triad which definitely contains a non-harmonic tone. The chord types were stored as collections of intervals as opposed to storing each individual chord (i.e. store D Major and G Major separately) to reduce on the amount of hard-coded variables which went into the code.

4.2 Chord Detection Techniques

Before any intervals could be calculated, it is important to first arrange the four notes into a suitable order. This order is very important if it is desired to get a list of intervals which look similar to those in ***Figure 4b***. It was decided to first arrange the notes in the order of a standard octave, starting at C and ending on B like so:

$$C – D – E – F – G – A – B$$

This means that if the values of four simultaneously occurring notes are for example, G, C, E and C, then they shall be arranged:

$$C – C – E – G$$

This would then allow the code to realise that this is a major triad with a doubled root. However this would not always work as if the four simultaneous notes were for example, B, F#, D# and F#, they would be arranged as:

$$D\# – F\# – F\# – B$$

This is not correct, as clearly this is also a major triad (B major with a doubled fifth) which cannot be calculated from this ordering. For this reason, for the initial pass of chord detection, the code would have to attempt to equate a list of intervals to a chord type four times, with the order changing each time, until a match is found. The match would be found of course when the notes are arranged as:

$$B – D\# - F\# - F\#$$

This essentially means that once the initial ordering has taken place, the index values of each note are shifted up by one each loop until a match can be found. This is the first attempt made by the code to match chords. At this stage a list is made the same length as the total number of notes in each voice. If a list of intervals is found to be the same as one of the pre-defined chords it is added

to the list in the correct position, otherwise it is filled in with a placeholder denoting that no match has yet been found.

The next technique which is applied is to check if any chords occur which are made up of only three notes. These happen fairly regularly, as it would be very unusual for an entire chorale to move in parallel motion. The same methodology as above has to be applied to iterate through different orderings of the notes. This same thing is then done however checking for chords with just two notes in them. It is important to check for three note chords before two, as to not only allow 2/3rds of a chord to be taken into account. Technically speaking the introduction of only two new notes does not necessarily mean a new chord is implied in the music, but for now they have been left in, to be corrected later by further filters.

After these two methods of determining chords, it was found that there was still some results which could not be matched. As mentioned in Section 4.1, there are often triads played with a non-harmonic tone included. Often this will be the result of a passing note which will resolve itself shortly into a note which remains consonant with the triad. In order for the code to detect this, it must perform interval calculations on all combinations of three notes out of four to see if a match has been found. If so, then the overall harmony at this point in music will be of whatever triad was found, and one note will of course be deemed as non-harmonic and shall be removed later.

The next check is to determine whether a chord which is not complete will resolve itself a few beats later in the score. This is very similar to the last method however its use is to categorise incomplete chords which complete shortly after having been played. An example would be having four simultaneous notes being G, F, A and E, with the A and E becoming a B and D respectively after less than a quarter beat has passed in the piece of music. The original rather confusing chord of G, A, F and E would now be G, F, B and D which could be determined that it is a G7 chord.

The final check applied is to see whether any chords are made up when only one new note is introduced. It is very rare that this is considered a standalone chord, as one part changing out of four would not make a huge difference musically. However there is one exception where the changed note may add the 7th to the previous chord. For now, all of these results are being stored, however as shall be explained in Section 4.4 not all chords which look like 7th chords are in fact harmonically correct in the music. The inclusion of a 7th to a chord depends largely on the key signature of the piece.

The result of performing all of these operations on the lists of notes given is simply one list of chordal structures, with the index value of the list indicating at what point in time they are implied in the music, similarly to how individual notes can also be pinpointed to a specific point in time in the music.

4.3 Removal of Non-Harmonic Tones

The final step is to be able to state which notes are consonant with their equivalent chordal structure, and which ones are dissonant. Since the chords were defined as lists of intervals as opposed to the specific notes which make each chord, it was necessary to construct each one from the list of intervals and the root. This means that in the code, a major triad exists, however a G Major triad has no meaning yet. This can simply be fixed as mentioned above by knowing the interval list and the root, as shown in the example below:

Root: G *Interval List: (P1, M3, P5)*

$$G \rightarrow P1 = G$$

$$G \rightarrow M3 = B$$
$$G \rightarrow P5 = D$$

With each chord being able to be reconstructed all that needs to be done is to compare the notes from all four voices to each equivalent chord and if that particular note is not present in the chord then that note value should be changed. In this case it is changed to “Remove” so that it is marked for removal from the score later.

Appendix A shows a tabled representation of how the computer analyses one of the Bach chorales (*'011007B_.xml'*). It can clearly be seen that each bar is divided into eight even segments as detailed in Section 3.2, with the notes from all four parts shown at which of these eight segments they are triggered. The resultant chords are shown below the lists of notes, and allow for an easy way to see which notes are deemed as non-harmonic.

Appendix B similarly shows a second example of a different Bach chorale (*'001405B_.xml'*). This time there are sixteen segments per bar, but the operation of the table is the same as above. The results of both of these examples are discussed in Section 4.4.

4.4 Ambiguity in Results

The results are relatively accurate but with some ambiguous details. The chords highlighted in green are definitely correct, which means all the resulting non-harmonic tones (highlighted in red) have also been removed successfully. The problems occur with the notes highlighted in blue.

For the most part, these are introductions of a 7th to a previously sounded chord which can be seen as either harmonic or non-harmonic. They have all been assumed as the former in this current version of the code. The reason for this is that in order to determine their harmonic inclusion to a chord, the current key signature of the piece is required. This is relatively simple to implement at first as the key is given at the start of any piece of music, so incorrect 7th chords can be fixed by first identifying the key of the piece. However it is not as simple as this, as most chorales tend to have some form of modulation in them, and as of yet there is no method to detect modulations in the score, so the inclusion of a filter to remove certain 7th chords at this time would probably just create more wrong answers than right.

Another way to weed out further incorrect results would be to check for cadences in the music. Usually Bach is consistent with his harmonic rhythm (i.e. the rate at which chord changes occur), so there is most often a new chord every quarter beat. However there can of course be exceptions to this rule, and there is often a temporary speeding up of this harmonic rhythm, which usually indicates that a cadence is occurring. Therefore it would make sense to implement a feature which would check for common cadences where the harmonic rhythm in the score had sped up. This is further explained in Section 6.1.

Finally, there were a few chords which the code failed to categorise at all, so more filters would have to be added to fix these. It seems that as further scores are analysed, the chord filtering technique is improved, as small nuances used by Bach are discovered.

5 Output as Musical Score

With all modifications performed on the original information to remove all notes that are considered to be non-harmonic, it is now required to make this data musically relevant once more by rescoring the music.

5.1 Compressing the Time Scale

Since the data is still in the exact same format as it was before manipulation, it is possible to correct the time scale in the music. Remember that in Section 3.3 a process was detailed to recover the original data after all the notes in the score had the time-slice operation applied to them. By using these flags it is possible to create a new list which contains only the information in which the value of each flag is equal to "Yes".

This returns a list of notes (with their associated information) the same length as the original number of notes present in the score. The only difference is that now some of the note values are simply marked as "Remove" so that the correct non-harmonic tones can be replaced in the score.

5.2 Write to XML File

In a musical score, you cannot just remove notes and not replace them with anything otherwise there will be bars with an uneven amount of beats. So to keep the new piece of music exactly like the original it is necessary to replace all these removed notes with rests. In order to be able to recreate the score, a new MusicXML file will be written from the new information by modifying the original file.

Fortunately it is possible in Minidom to entirely replace an entire node (tag) with a new user-defined one. So before this can be done it has to be shown what a rest tag would look like in a MusicXML file. ***Figure 5a*** shows the direct replacement that can be made to the pitch tag.

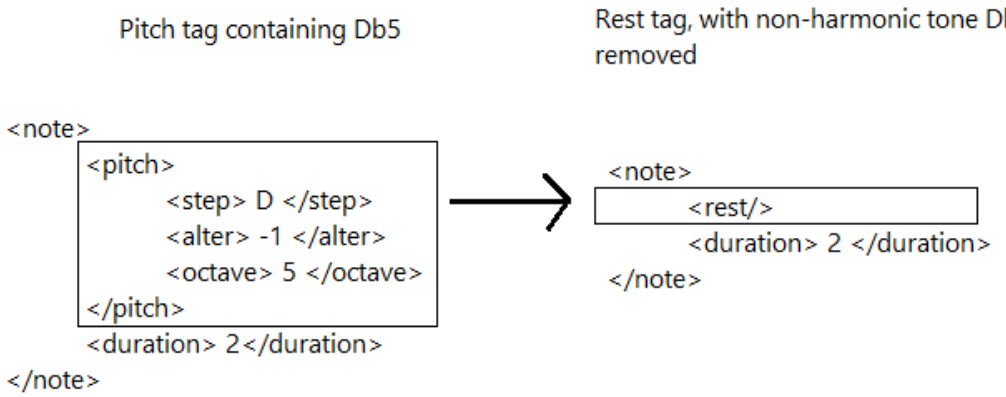


Figure 5a - *A typical note structure for a MusicXML file denoting a Db5 note with duration 2(left). The entire pitch tag can be replaced by a rest tag, leaving the duration the same (right).*

Since the length of notes in both the XML file and the list of modified data are the same, it is

possible to step through each one iteratively checking to see if there is any occurrence of the note value being equal to “Remove”. If the nth note in the list conforms to this check, then the nth note of the XML file will have its pitch tag removed and replaced with a new rest tag. The file is then saved under a new filename so as to not delete the original MusicXML file so that it exists for comparison.

5.3 Output via Lilypond

In order to actually view this MusicXML file as a sheet of music, the open-source notation software Lilypond [3] was used. This can simply take an XML file as an input and convert it into its own syntax, which can then be used to create a PDF file of the actual sheet music. These instructions can be triggered through Ubuntu's command Terminal by typing:

musicxml2ly –nd –nrp –npl –no-beaming file.xml

All the additional commands after the initial *musicxml2ly* are just highly recommended [12] settings to allows Lilypond to perform the best conversion possible. This creates a Lilypond file (.ly) which is in the notation softwares own unique syntax. Then one further command turns this syntax into a PDF file:

lilypond file.ly

Example output files are included in **Appendices C – F**, which show the two original example scores from Section 4.3, and the new versions of the score which have had their non-harmonic tones removed and replaced with rests.

This completes the overall design of the system, and a flow diagram for the software can be seen in **Appendix G**.

6 Further Considerations & Improvements

Although not 100% accurate, it can clearly be seen that a score of music can be taken as an input and then outputted with most of the non-harmonic tones removed. Further considerations and future improvements which can be implemented are stated in this section.

6.1 Key Signatures & Modulation

Modulation was touched upon in Section 5.4, and it has become clear that it is the key to achieving a fully accurate removal of non-harmonic tones. The problem is finding a method which can be used to detect modulations. The key signature at the start of the piece is stated in a MusicXML file however it will not be the key which the score is in for every single bar. There is usually always at least one phrase with some kind of modulation present. Knowing the modulations throughout a score would enable the accuracy of checking for 7th chords and cadences.

For example if the key is known, then the scale degrees will be known. These scale degrees allow for an ordered system of the most common 7th chords in the current key. [16] This would obviously establish a method for labelling 7th notes are either harmonic or non-harmonic.

6.2 Harmonic Rhythm & Cadences

The consistency of Bach's harmonic rhythm mentioned in Section 5.4 can also be a key factor in correcting any inaccuracies in the chord classification system. This is perhaps best described with an example as shown in ***Figure 6a***.



Bar 8 :									
Soprano	E	-	D	-	C#	-	C#	-
Alto	G#	-	A	B	A	-	A	-
Tenor	B	-	F#	E	E	-	E	-
Bass	E	-	F#	G#	A	-	A	G
Chord	E	-	D	E7	A	-	A	A7

Bar 9 :								
Soprano	D	-	D	-	E	-	E	-
Alto	A	G	F#	-	B	A	G#	-
Tenor	F#	E	D	C#	B	-	E	-
Bass	F#	-	B	A	G#	-	C#	B
Chord	D	-	Bm	DM7	E	-	C#m	-

Figure 6a *– Bars 8 and 9 from 011007B_.xml (top) and the tabled representation of each bar showing their implied harmony (bottom).*

Both of these bars have a section of three consecutive chords between the third and fifth segments. However, analysing by hand shows that only one of them is actually correct. In Bar 8, the progression goes D → E7 → A, which can very much be considered a classic cadence, however the one in Bar 9 goes Bm → DM7 → E, which would not be considered to be a commonly occurring cadence. For this reason with this analysis, the C# and A notes in Bar 9 should be marked as non-harmonic tones and removed from the score. The fact that three new notes are introduced on the E7 also give it more legitimacy as a standalone chord compared to the DM7, which only introduces two new notes on the beat.

However, implementing this system would once again require the knowledge of all modulations throughout the piece as determining cadences also relies on knowing each of the scale degrees. So it can be clear to see that the most important part of research which has to be put into this system next is a method to identify modulations.

7 Conclusion

The results show that it will indeed be possible to fully complete the aims of the project with future improvements. However that is not to say that the results themselves are not positive or successful, as it can indeed identify the majority of chordal structures and non-harmonic tones present in a music score. Improvements will continue to be made as more scores are analysed, with the chord filtering procedure becoming more and more fine tuned to the required specification.

The number of notes removed could easily be improved by implementing a system to detect incorrect 7th chords, and by checking for cadences in areas where the harmonic rhythm has noticeably increased, however these would produce false results without any knowledge of where modulations have taken place.

Therefore in conclusion, for this project to continue it is necessary to research and find a way to detect these modulations and apply this to the system in order to provide the most accurate results possible. It is believed that doing this will be able to complete the task at hand and satisfy the project aim of removing non-harmonic tones from a music score.

Appendices

Appendix A – Result of chord classification filters on the Bach chorale "*011007B_.xml*". Shows 8 segments per bar (apart from first and last as there is an anacrusis present in the score) with the notes triggered by each voice at the correct segment in time. Chords underneath show overlying harmony at the current time in the score. Green denotes a chord which is correct. Red denotes a note which is considered non-harmonic and Blue indicates notes and chords which are currently debatable to the system, they are of course still possible to be solved by hand.

Appendix B – Result of chord classification filters on the Bach chorale "*001405B_.xml*". Shows 16 segments per bar (with an anacrusis present once more). The same colouring scheme in Appendix A is applied here.

Appendix C – The original un-edited score produced from the original XML file "*011007B_.xml*".

Appendix D – The modified score produced from the XML file "*011007B_.xml*" with the non-harmonic tones removed.

Appendix E – The original un-edited score produced from the original XML file "*001405B_.xml*".

Appendix F – The modified score produced from the XML file "*001405B_.xml*" with the non-harmonic tones removed.

Appendix G – The software flow diagram.

Appendix H – CD containing all code, PDF copy of report and the original XML files from the two examples.

Appendix A

Bar 0 :		
Soprano	B	C#
Alto	F#	-
Tenor	D	E
Bass	B	-
Chord	Bm	-

Bar 1 :								
Soprano	D	-	C#	-	B	-	B	C#
Alto	B	-	A#	-	F#	-	F#	-
Tenor	F#	-	F#	E	D	-	D	E
Bass	B	-	F#	-	B	-	B	-
Chord	Bm	-	F#	F#7	Bm	-	Bm	-

Bar 2 :								
Soprano	D	-	C#	-	B	-	F#	-
Alto	B	-	A#	-	F#	-	F#	G
Tenor	F#	-	F#	E	D	-	A	B
Bass	B	-	F#	-	B	-	D	-
Chord	Bm	-	F#	F#7	Bm	-	D	G7

Bar 3 :								
Soprano	E	-	D	-	C#	-	C#	-
Alto	A	-	F#	-	F#	-	F#	-
Tenor	C#	-	B	-	A#	-	A#	-
Bass	A	-	B	-	F#	-	F#	-
Chord	A	-	Bm	-	F#	-	F#	-

Bar 4 :								
Soprano	D	-	D	-	E	-	E	-
Alto	F#	-	F#	-	B	-	A	-
Tenor	B	C	D	-	D	-	C#	-
Bass	B	-	B	A	G	E	A	G
Chord	Bm	-	Bm	-	Em7	-	A	A7

Bar 5 :								
Soprano	F#	-	F#	-	E	-	D	-
Alto	A	-	A	B	C#	-	F#	-
Tenor	D	-	D	-	C#	-	B	-
Bass	F#	E	D	C#	B	A#	B	-
Chord	D	-	D	-	C#m	-	Bm	-

Bar 6 :								
Soprano	C#	-	-	-	B	-	F#	-
Alto	F#	-	-	E	-	D	B	-
Tenor	B	-	A#	-	F#	-	D	-
Bass	F#	-	-	-	B	-	B	-
Chord	F#	-	F#	F#7	Bm	-	Bm	-

Bar 7 :								
Soprano	E	-	D	-	C#	-	F#	-
Alto	B	A#	B	-	A#	-	B	A
Tenor	E	-	F#	-	F#	-	F#	-
Bass	C#	-	D	E	F#	-	D	-
Chord	C#m	-	Bm	-	F#	-	Bm	-

Bar 8 :								
Soprano	E	-	D	-	C#	-	C#	-
Alto	G#	-	A	B	A	-	A	-
Tenor	B	-	F#	E	E	-	E	-
Bass	E	-	F#	G#	A	-	A	G
Chord	E	-	D	E7	A	-	A	A7

Bar 9 :								
Soprano	D	-	D	-	E	-	E	-
Alto	A	G	F#	-	B	A	G#	-
Tenor	F#	E	D	C#	B	-	E	-
Bass	F#	-	B	A	G#	-	C#	B
Chord	D	-	Bm	DM7	E	-	C#m	-

Bar 10 :								
Soprano	F#	-	F#	-	E	-	D	-
Alto	C#	-	B	A#	B	-	B	-
Tenor	E	-	D	C#	B	C#	D	E
Bass	A#	-	D	-	G#	A	B	-
Chord	F#7	-	Bm	-	E	-	Bm	-

Bar 11 :								
Soprano	C#	-	-	-	B	-		
Alto	B	-	A#	-	F#	-		
Tenor	F#	-	-	E	D#	-		
Bass	F#	-	-	-	B	-		
Chord	F#	-	F#	F#7	B	-		

Appendix B

Bar 0 :																
Soprano	G	-	A	-												
Alto	D	-	-	-												
Tenor	Bb	-	C	-												
Bass	G	-	-	-												
Chord	Gm	-	-	-												

Bar 1 :																
Soprano	Bb	-	-	-	B	-	-	-	C	-	-	-	D	-	-	-
Alto	G	-	-	-	G	-	-	-	G	-	-	-	G	-	-	-
Tenor	D	-	-	-	D	-	-	-	Eb	-	-	-	D	-	-	-
Bass	G	-	Ab	-	G	-	F	-	Eb	-	D	-	C	-	B	-
Chord	Gm	-	-	-	G	-	G7	-	Cm	-	-	-	G	-	-	-

Bar 2 :																
Soprano	C	-	-	-	B	-	-	-	C	-	-	-	C	-	-	-
Alto	Eb	-	-	-	D	-	-	-	C	-	-	-	G	-	A	-
Tenor	G	-	G	-	-	-	F	-	Eb	-	-	-	Eb	-	-	-
Bass	C	-	Eb	-	G	-	-	-	C	-	-	-	C	-	-	-
Chord	Cm	-	Cm	-	G	-	G7	-	Cm	-	-	-	Cm	-	-	-

Bar 3 :																
Soprano	Bb	-	A	-	G	-	A	-	Bb	-	C	-	D	-	-	-
Alto	Bb	-	Bb	-	-	-	C	-	D	-	Eb	-	F	-	G	-
Tenor	F	-	-	-	G	-	-	-	F	-	Eb	-	D	-	Bb	-
Bass	D	-	-	-	Eb	-	-	-	D	-	C	-	Bb	-	Eb	-
Chord	Bb	-	BbM7	-	Eb	-	-	-	Bb	-	Cm	-	Bb	-	EbM7	-

Bar 4 :																
Soprano	C	-	Bb	-	C	-	-	-	Bb	-	-	-	G	-	A	-
Alto	A	-	Bb	-	-	-	A	-	F	-	-	-	Eb	-	-	-
Tenor	F	-	-	-	G	-	F	Eb	D	-	-	-	Bb	-	C	-
Bass	Eb	-	D	-	Eb	-	F	-	Bb	-	-	-	Eb	-	-	-
Chord	F7	-	Bb	-	Cm7	-	F	F7	Bb	-	-	-	Eb	-	-	-

Bar 5 :																
Soprano	Bb	-	-	-	B	-	-	-	C	-	-	-	D	-	-	-
Alto	F	-	-	-	G	-	-	-	G	-	-	-	G	-	-	-
Tenor	D	-	-	-	D	-	-	-	Eb	-	-	-	D	-	-	-
Bass	G	-	Ab	-	G	-	F	-	Eb	-	D	-	C	-	B	-
Chord	Gm7	-	Bb7	-	G	-	G7	-	Cm	-	-	-	G	-	-	-

Bar 6 :																
Soprano	C	-	-	-	B	-	-	-	C	-	-	-	C	-	-	-
Alto	Eb	-	-	-	D	-	-	-	C	-	-	-	G	-	A	-
Tenor	G	-	G	-	-	-	F	-	Eb	-	-	-	Eb	-	-	-
Bass	C	-	Eb	-	G	-	-	-	C	-	-	-	C	-	-	-
Chord	Cm	-	-	-	G	-	G7	-	Cm	-	-	-	Cm	-	-	-

Bar 7 :																
Soprano	Bb	-	A	-	G	-	A	-	Bb	-	C	-	D	-	-	-
Alto	Bb	-	Bb	-	-	-	C	-	D	-	Eb	-	F	-	G	-
Tenor	F	-	-	-	G	-	-	-	F	-	Eb	-	D	-	Bb	-
Bass	D	-	-	-	E	-	-	-	D	-	C	-	Bb	-	Eb	-
Chord	Bb	-	BbM7	-	Eb	-	-	-	Bb	-	Cm	-	Bb	-	EbM7	-

Bar 8 :																
Soprano	C	-	Bb	-	C	-	-	-	Bb	-	-	-	C	-	-	-
Alto	A	-	Bb	-	-	-	Ab	-	F	-	-	-	F	-	Eb	-
Tenor	F	-	-	-	G	-	F	Eb	D	-	-	-	C	-	-	-
Bass	Eb	-	D	-	Eb	-	F	-	Bb	-	-	-	A	-	-	-
Chord	F7	-	Bb	-	Cm7	-	F	F7	Bb	-	-	-	F	-	-	-

Bar 9 :																
Soprano	D	-	-	-	C	-	Bb	-	A	-	-	-	Bb	-	-	-
Alto	D	-	E	-	F	-	G	-	C	-	-	-	F	-	-	-
Tenor	C	-	B	-	A	-	G	-	F	-	-	-	-	-	G	-
Bass	Bb	-	C	-	D	-	E	-	F	-	Eb	-	D	-	-	-
Chord	Dm7	-	-	-	Dm7	-	Em	-	F	-	F7	-	Bb	-	Gm7	-

Bar 10 :																
Soprano	C	-	Bb	-	A	-	-	-	G	-	-	-	G	-	-	-
Alto	Eb	-	-	-	A	-	D	-	D	-	-	-	D	-	-	-
Tenor	A	-	G	-	-	-	F#	-	Bb	-	-	-	B	-	-	-
Bass	C	-	-	-	D	-	-	-	G	-	-	-	G	-	F	-
Chord	Cm	-	Cm7	-	?	-	D	-	Gm	-	-	-	G	-	G7	-

Bar 11 :																
Soprano	C	-	-	-	B	-	-	-	C	-	-	-	D	-	C	-
Alto	G	-	Ab	-	G	-	F	-	Eb	-	-	-	D	-	F#	-
Tenor	C	-	-	-	D	-	-	-	G	-	-	-	Bb	-	A	-
Bass	Eb	-	-	-	D	-	-	-	C	-	-	-	G	-	-	-
Chord	Cm	-	-	-	G	-	G7	-	Cm	-	-	-	Gm	-	?	-

Bar 12 :																
Soprano	Bb	-	A	-	G	-	-	-	F	-	-	-	F	-	-	-
Alto	G	-	D	-	-	-	C#	-	D	-	-	-	D	-	-	-
Tenor	G	-	A	-	Bb	-	A	-	A	-	-	-	Bb	-	-	-
Bass	G	-	F	-	E	-	A	-	D	-	-	-	Bb	-	A	-
Chord	G	-	Dm	-	?	-	-	-	Dm	-	-	-	Bb	-	BbM7	-

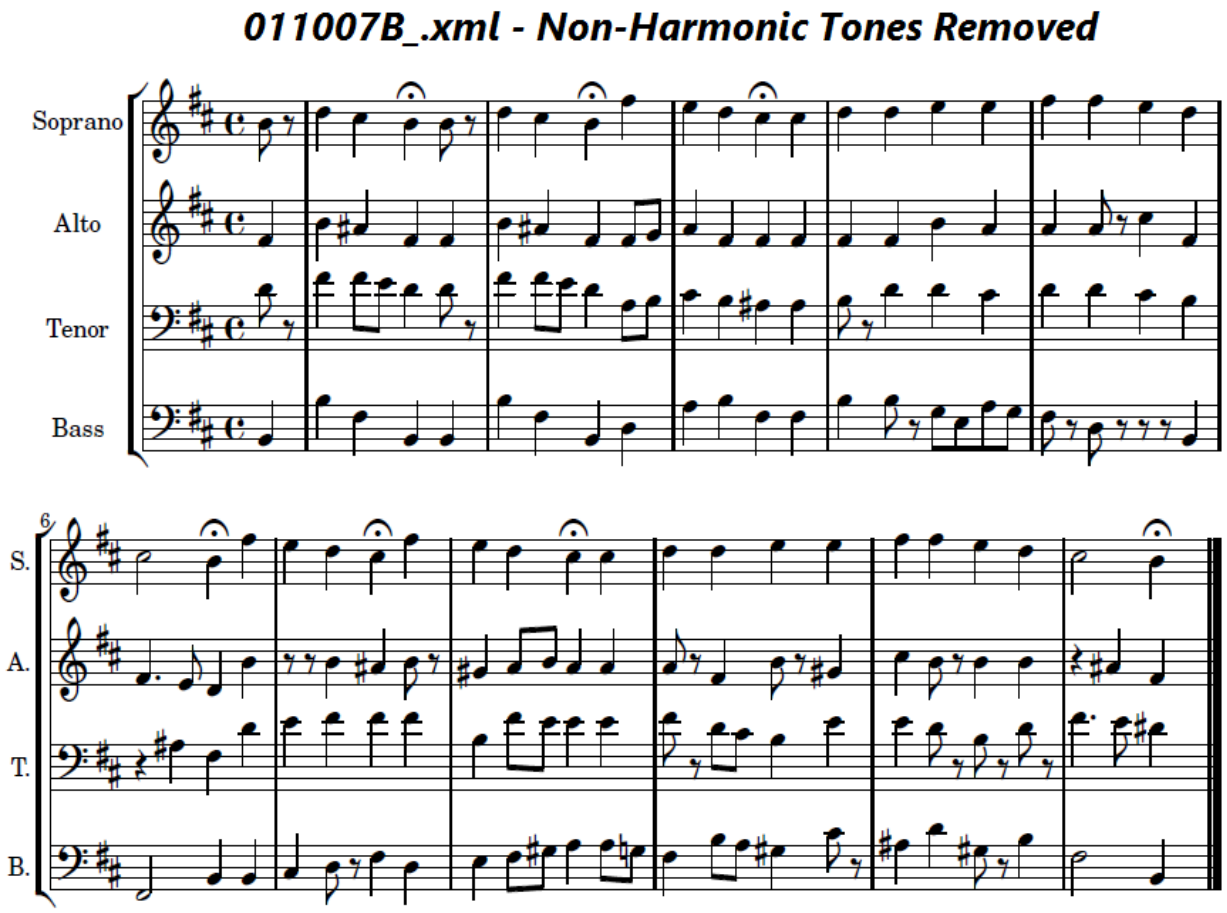
Bar 13 :																
Soprano	Bb	-	-	-	C	-	-	-	D	-	-	-	C	-	Bb	-
Alto	D	-	-	-	G	-	-	-	F#	-	D	-	G	-	-	-
Tenor	Bb	-	A	-	G	-	-	-	A	-	Bb	C	D	-	-	-
Bass	G	-	F	-	Eb	-	D	-	C	-	Bb	-	A	-	-	-
Chord	Gm	-	BbM7	-	Cm	-	-	-	D7	-	Bb	-	Gm	-	-	-

Bar 14 :																
Soprano	A	-	G	-	A	-	-	-	G	-	-	-				
Alto	G	-	-	-	F#	-	-	-	D	-	-	-				
Tenor	D	-	-	-	-	-	C	-	B	-	-	-				
Bass	D	-	-	-	-	-	-	-	G	-	-	-				
Chord	G	-	-	-	D	-	D7	-	G	-	-	-				

Appendix C



Appendix D


011007B_.xml - Non-Harmonic Tones Removed
Soprano
Alto
Tenor
Bass
6
S.
A.
T.
B.

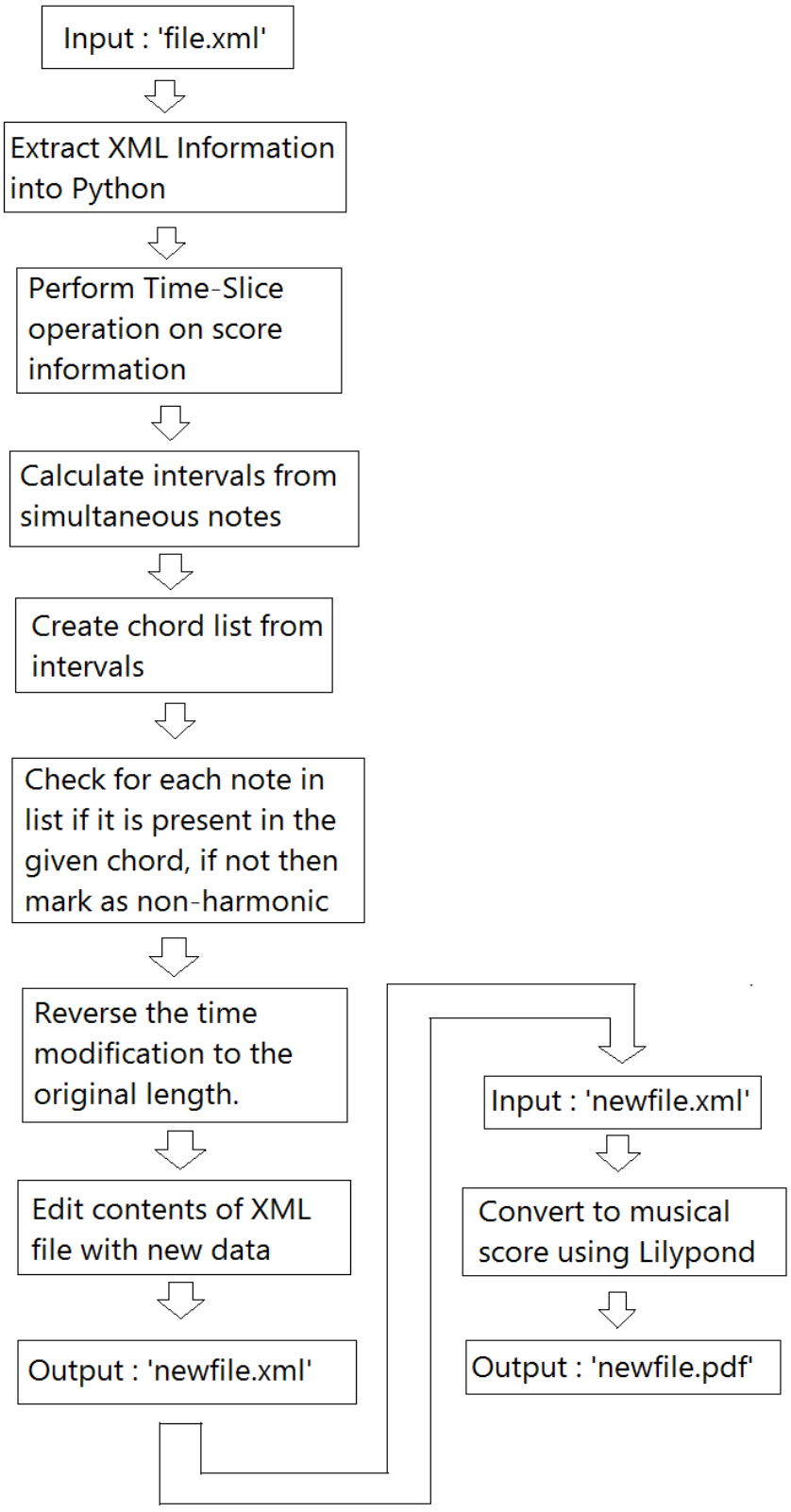
Appendix E



Appendix F


001405B_.xml - Non-harmonic Tones Removed
Soprano
Alto
Tenor
Bass
5
S.
A.
T.
B.
10
S.
A.
T.
B.

Appendix G


Input : 'file.xml'
Extract XML Information into Python
Perform Time-Slice operation on score information
Calculate intervals from simultaneous notes
Create chord list from intervals
Check for each note in list if it is present in the given chord, if not then mark as non-harmonic
Reverse the time modification to the original length.
Edit contents of XML file with new data
Output : 'newfile.xml'
Input : 'newfile.xml'
Convert to musical score using Lilypond
Output : 'newfile.pdf'

Appendix H

All the following included on CD:

- *Main.py* – The main program which contains the control loop of all the functions
- *TimeManipulation.py* – A separate module containing a class to perform time operations on the data
- *Parser.py* – A separate module containing a class to extract data from XML files and also write data to new XML files.
- *ModifySpoff.py* – The Spoff calculations required to carry out all the above, they have been modified slightly.
- *011007B_.xml* – XML file for Bach chorale used in examples
- *001405B_.xml* – XML file for Bach chorale used in examples

Bibliography

[1] - Beautiful Soup Library http://www.crummy.com/software/BeautifulSoup/bs4/doc/

[2] - ElementTree XML API https://docs.python.org/2/library/xml.etree.elementtree.html

[3] - Lilypond Music Engraving Software http://lilypond.org/

[4] - lxml - XML and HTML with Python http://lxml.de/

[5] - Minidom - Minimal DOM Implementation
https://docs.python.org/2/library/xml.dom.minidom.html

[6] - MusicXML File Exchange Format http://www.musicxml.com/

[7] - PostgreSQL Database Software http://www.postgresql.org/

[8] - Python Default Libraries https://docs.python.org/2.7/library/

[9] - Python Dictionaries https://docs.python.org/2/tutorial/datastructures.html

[10] - Python High-Level Programming Language https://www.python.org/

[11] - XML (extensible markup language) http://www.w3.org/XML/

[12] - Bjuhr, P. (2013). Importing Music Into Lilypond. Available:
http://lilypondblog.org/2013/11/importing-music-into-lilypond/. Last accessed 3rd Apr 2014.

[13] - Buszin, W (1952). 101 Chorales Harmonized by J.S. Bach. Van Nuys, CA: Alfred.

[14] - Finn, J. Chord Speller. Available: http://www.joefinn.net/html/chord_speller.html. Last accessed 2nd Apr 2014.

[15] - Franklin, D (1989). Bach Studies. London: Cambridge University Press. p292.

[16] - Kostka & Payne (1995). Tonal Harmony, p.225. Third Edition.

[17] - Pullinger, S(2010) A system for the analysis of musical data. PhD thesis, University of Glasgow. Available at http://theses.gla.ac.uk/2133/

[18] - Sibelius Academy. (2006). Non-Harmonic Notes. Available:
http://www2.siba.fi/muste1/index.php?id=66&la=en. Last accessed 1st Apr 2014